ShopEase - Microservices-Based E-Commerce Management System

# Monolithic vs Microservices Architecture

Document 1 - Comparison

Prepared by: Dineshkumar S | Day 1 Deliverable

## 1. What is a Monolithic Architecture?

In a monolithic application, the whole application is built as one single unit. All features such as login, products, orders and payments are in one codebase. They are deployed together as one package and they usually use one shared database.

Example: If ShopEase was built as a monolith, there would be one big FastAPI project. Registration, product list, orders and payments would all be inside the same project and would all read and write the same database.

### Advantages of a monolith

- Simple to build and easy to start, because there is only one project.
- Easy to test and debug on a developer's laptop.
- Only one deployment is needed.
- No network calls between features, so it is fast inside.

### Disadvantages of a monolith

- If one part has a bug or crashes, the whole application can go down.
- The whole application must be scaled together, even if only one feature (like product search) is busy.
- A small change needs the full application to be tested and deployed again.
- The code becomes big and hard to understand as the project grows.
- The whole project is tied to one technology and one shared database.

## 2. What is a Microservices Architecture?

In a microservices architecture, the application is split into small services. Each service does one job, runs on its own, has its own database, and can be deployed and scaled separately. The services talk to each other using REST APIs over HTTP.

Example: ShopEase is split into the Authentication, User, Product, Order and Payment services, with an API Gateway in front of them.

### Advantages of microservices

- Each service can be scaled alone. For example, only the Product Service can be scaled during a sale.
- A failure in one service does not stop the other services.
- Different teams can work on different services at the same time.
- Each service can be updated and deployed without touching the others.
- Each service can use the technology and database that suits it best.

### Disadvantages of microservices

- The system is more complex because there are many services to manage.
- Services talk over the network, so there can be delays and failures.
- Debugging is harder because one request goes through many services.
- Keeping data correct across different databases is harder.
- Deployment needs tools like Docker and Docker Compose.

## 3. Comparison Table

| Point | Monolithic | Microservices |
|---|---|---|
| Structure | One single codebase | Many small independent services |
| Database | One shared database | Separate database for each service |
| Scalability | Whole application scales together | Each service scales on its own |
| Deployment | One deployment for everything | Each service is deployed separately |
| Fault isolation | One bug can bring down everything | A failure stays inside one service |
| Technology | Usually one technology for all | Each service can use a different technology |
| Team work | Teams depend on each other | Teams work independently |
| Testing | Easy at the start, slow when big | Each service is small and easy to test; integration testing is harder |
| Complexity | Simple to begin, complex later | More complex from the beginning |
| Communication | Function calls inside the code | REST API calls over the network |

## 4. Why Microservices Suit Large Enterprise Applications

Large applications like online shopping sites have many features, many users and many developers. Microservices help in these ways:

- Scalability: During a sale, thousands of people browse products, but fewer people pay. The Product Service can be scaled up without scaling the Payment Service.
- Reliability: If the Payment Service has a problem, customers can still log in and browse products.
- Faster releases: A team can update the Order Service without waiting for the other teams.
- Easy maintenance: Each service is small, so a new developer can understand it quickly.
- Security: Sensitive data like payments can be kept in a separate service with stricter rules.

## 5. Conclusion

A monolith is a good choice for small projects because it is simple. For a large enterprise application like ShopEase, microservices are a better choice because they give better scaling, better fault isolation and faster teamwork. The price is extra complexity, which we handle using an API Gateway, Docker and clear service responsibilities.

ShopEase - Microservices-Based E-Commerce Management System

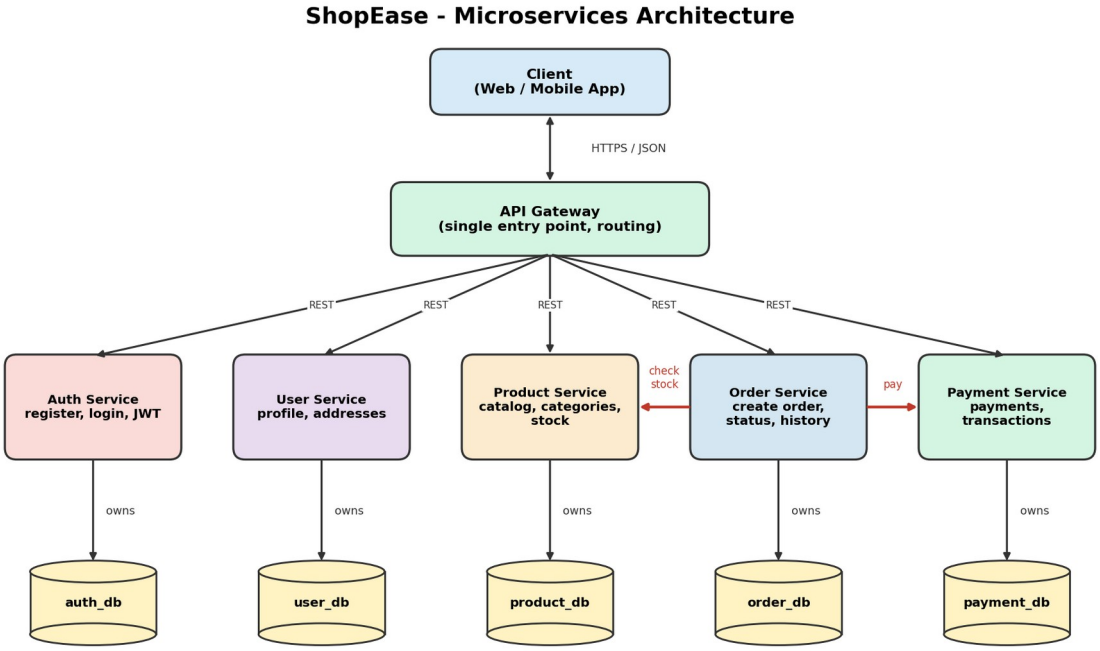
# Service Responsibilities

Document 2 - Responsibilities of each microservice

Prepared by: Dineshkumar S | Day 1 Deliverable

## 1. Overview

ShopEase has six microservices. Each one has a clear job. A service does only its own job and asks other services through APIs when it needs something else. The diagram below shows the full system.




*Black arrows: client requests routed by the gateway. Red arrows: Order Service calling other services (REST). Each service has its own database.*

## 2. Authentication Service

Purpose: Takes care of who the user is and whether they are allowed to enter the system.

### Responsibilities

- Register new users (email and password).
- Store passwords safely as hashed values, never as plain text.
- Login and check the email and password.
- Create a JWT (JSON Web Token) after a successful login.
- Verify JWT tokens when other services ask.

Does not do: It does not store profile details, addresses, products or orders.

Database: auth_db (users, credentials)

## 3. User Service

Purpose: Manages the profile information of customers.

### Responsibilities

- View and update the user profile (name, phone number).
- Add, update, list and delete delivery addresses.

Does not do: It does not handle login or passwords.

Database: user_db (profiles, addresses)

## 4. Product Service

Purpose: Manages everything about the items we sell.

### Responsibilities

- Product catalog: add, view, update and delete products.
- Categories: group products such as Mobiles, Laptops and Clothes.
- Inventory: keep the stock count of every product.
- Reduce stock when an order is placed and increase it again if the order fails.

Does not do: It does not create orders or take payments.

Database: product_db (products, categories, inventory)

## 5. Order Service

Purpose: Handles the customer's orders from start to finish.

### Responsibilities

- Create a new order with a list of items.
- Check product availability by calling the Product Service.
- Start the payment by calling the Payment Service.
- Keep the order status: PENDING, PAID, FAILED, CANCELLED.
- Show order history and order details for a customer.

Does not do: It does not store product details or payment details itself. It asks the other services.

Database: order_db (orders, order_items)

## 6. Payment Service

Purpose: Handles money related records.

### Responsibilities

- Create a payment record for an order.
- Keep the transaction history.
- Keep the payment status: SUCCESS, FAILED, PENDING.

Does not do: It does not create orders or change stock.

Database: payment_db (payments, transactions)

## 7. API Gateway

Purpose: It is the single entry point for all client requests.

### Responsibilities

- Receive every request from the client (web or mobile app).
- Route the request to the correct service, for example /products goes to the Product Service.
- Check the JWT token before sending protected requests forward.
- Hide the internal services, so the client only knows one address.

Database: The gateway has no database of its own.

## 8. Summary Table

| Service | Main job | Database |
|---|---|---|
| Authentication | Register, login, JWT | auth_db |
| User | Profile and addresses | user_db |
| Product | Catalog, categories, inventory | product_db |
| Order | Create orders, status, history | order_db |
| Payment | Payments, transactions, status | payment_db |
| API Gateway | Routing, single entry point | None |

ShopEase - Microservices-Based E-Commerce Management System

# Database Ownership

Document 3 - Database per service

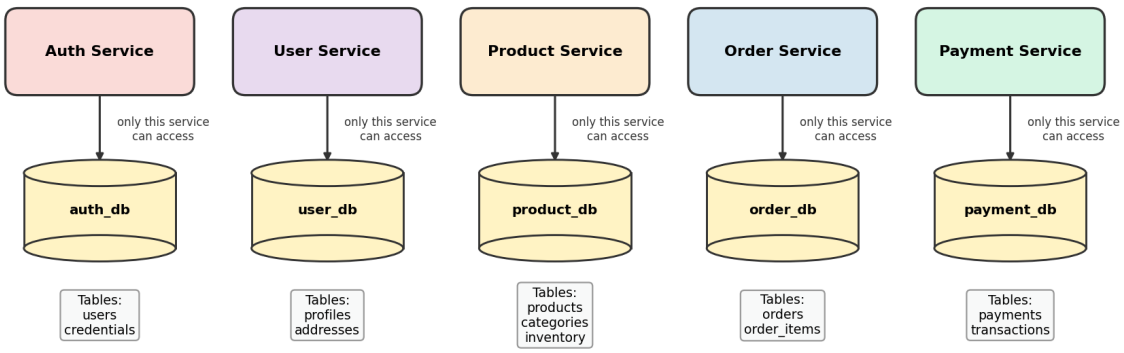
Prepared by: Dineshkumar S | Day 1 Deliverable

## 1. Database Per Service

In microservices, every service has its own database. Only that service is allowed to read or write it. Other services cannot connect to it. If they need the data, they must ask the owner service through its API.

In ShopEase we use MySQL, and each service has its own database: auth_db, user_db, product_db, order_db and payment_db. The API Gateway has no database.

**ShopEase - Database Ownership (one database per service)**




***No service connects to another service's database. Data is shared only through REST APIs.***

## 2. Tables in Each Database

| Database | Owner | Main tables |
|---|---|---|
| auth_db | Authentication Service | users (id, email, password_hash, created_at) |
| user_db | User Service | profiles (id, user_id, name, phone); addresses (id, user_id, line1, city, pincode) |
| product_db | Product Service | categories (id, name); products (id, name, price, category_id); inventory (product_id, stock_quantity) |
| order_db | Order Service | orders (id, user_id, total_amount, status, created_at); order_items (id, order_id, product_id, quantity, price) |
| payment_db | Payment Service | payments (id, order_id, amount, status); transactions (id, payment_id, reference, created_at) |

## 3. Database Isolation and Service Independence

- Isolation: A problem in one database, such as a slow query or a crash, does not affect the other services.
- Independent changes: A service can change its tables without breaking other services.
- Independent scaling: A busy database, like product_db, can be scaled alone.
- Technology freedom: Each service can choose the database type that suits it best.
- Security: Sensitive data, such as passwords and payments, stays in its own protected database.

## 4. Why Not Access Another Service's Database Directly?

Suppose the Order Service reads product_db directly to check stock. This causes problems:

- Tight coupling: If the Product team renames a column, the Order Service breaks without warning.
- No business rules: The Product Service has rules about stock. Reading the database directly skips those rules.
- Security risk: Many services holding passwords to the same database is unsafe.
- Hard to scale or replace: We cannot change or move product_db easily because other services depend on it.

The correct way is to call the Product Service API, for example GET /products/5. The Product Service checks its own rules and returns only the data that is needed.

## 5. Data Across Services

Because the data is split, an order only stores the product_id and the user_id, not the full product or user details. When details are needed, the Order Service calls the Product Service or the User Service. The price is saved inside order_items at the time of purchase, so old orders remain correct even if the price changes later.

## 6. Conclusion

Database per service keeps the services independent and safe. The cost is that data must be fetched through APIs and some consistency work is needed, but this is the standard approach in microservices.

ShopEase - Microservices-Based E-Commerce Management System

# REST API and HTTP Methods Notes

Document 4 - With examples

Prepared by: Dineshkumar S | Day 1 Deliverable

## 1. What is a REST API?

A REST API lets two programs talk using HTTP. The client sends a request to a URL (an endpoint) and the server sends back a response, usually in JSON format. In ShopEase, the client talks to the API Gateway and the services talk to each other in the same way.

## 2. HTTP Methods

| Method | Use | ShopEase example |
|---|---|---|
| GET | Read data | GET /products - list all products |
| POST | Create new data | POST /orders - create a new order |
| PUT | Replace the whole resource | PUT /users/me - update the full profile |
| PATCH | Change only some fields | PATCH /products/5/stock - change only the stock |
| DELETE | Remove data | DELETE /users/me/addresses/3 - delete an address |

## 3. Request and Response with JSON

JSON is a simple text format with keys and values. A request body is the data sent by the client. A response body is the data returned by the server.

### GET example - get one product

```
GET /products/5

Response 200 OK
{
  "id": 5,
  "name": "Wireless Mouse",
  "price": 799.0,
  "stock": 40
}
```

### POST example - register a user

```
POST /auth/register

Request body
{
  "email": "user@example.com",
  "password": "Secret@123"
}

Response 201 Created
{
  "id": 1,
```

```
  "email": "user@example.com"
}
```

### PUT example - update the full profile

```
PUT /users/me

Request body
{
  "name": "Ravi Kumar",
  "phone": "9876543210"
}

Response 200 OK
{
  "name": "Ravi Kumar",
  "phone": "9876543210"
}
```

### PATCH example - change only the stock

```
PATCH /products/5/stock

Request body
{
  "stock": 35
}

Response 200 OK
{
  "id": 5,
  "stock": 35
}
```

### DELETE example - delete an address

```
DELETE /users/me/addresses/3

Response 204 No Content (no body)
```

## 4. HTTP Status Codes

| Code | Name | Meaning |
|---|---|---|
| 200 | OK | The request worked and data is returned. |
| 201 | Created | A new resource was created (for example a new order). |
| 204 | No Content | The request worked and there is nothing to return (for example after DELETE). |
| 400 | Bad Request | The request is wrong or incomplete. |
| 401 | Unauthorized | The user is not logged in or the JWT is missing or invalid. |
| 403 | Forbidden | The user is logged in but is not allowed to do this. |
| 404 | Not Found | The resource does not exist. |
| 409 | Conflict | The request clashes with existing data (for example the |

| Code | Name | Meaning |
|---|---|---|
| | | email is already registered). |
| 422 | Unprocessable Entity | The data format is invalid. FastAPI and Pydantic return this for validation errors. |
| 500 | Internal Server Error | Something went wrong inside the server. |
| 503 | Service Unavailable | The service is down or too busy (used when a dependent service is not reachable). |

## 5. Example Endpoints for Each Service

### Authentication Service

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /auth/register | Register a new user |
| POST | /auth/login | Login and get a JWT token |
| GET | /auth/verify | Verify a token |

### User Service

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /users/me | View my profile |
| PUT | /users/me | Update my profile |
| POST | /users/me/addresses | Add an address |
| GET | /users/me/addresses | List my addresses |
| DELETE | /users/me/addresses/{id} | Delete an address |

### Product Service

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /products | List all products |
| GET | /products/{id} | Get one product |
| POST | /products | Add a product |
| PUT | /products/{id} | Update a product |
| PATCH | /products/{id}/stock | Update the stock |
| DELETE | /products/{id} | Delete a product |
| GET | /categories | List categories |

### Order Service

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /orders | Create an order |
| GET | /orders | Order history of the logged-in user |
| GET | /orders/{id} | Get one order |
| PATCH | /orders/{id}/status | Update the order status |

### Payment Service

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /payments | Make a payment for an order |
| GET | /payments/{id} | Get one payment |
| GET | /payments/order/{order_id} | Get the payment of an order |
| GET | /transactions | Transaction history |

## 6. Testing the APIs

FastAPI gives Swagger UI automatically at /docs, where every endpoint can be tried from the browser. Postman can also be used to send requests, add the JWT token in the Authorization header (Bearer token) and check the responses.

ShopEase - Microservices-Based E-Commerce Management System

# Service-to-Service Communication Workflow

Document 5 - Order flow and error handling

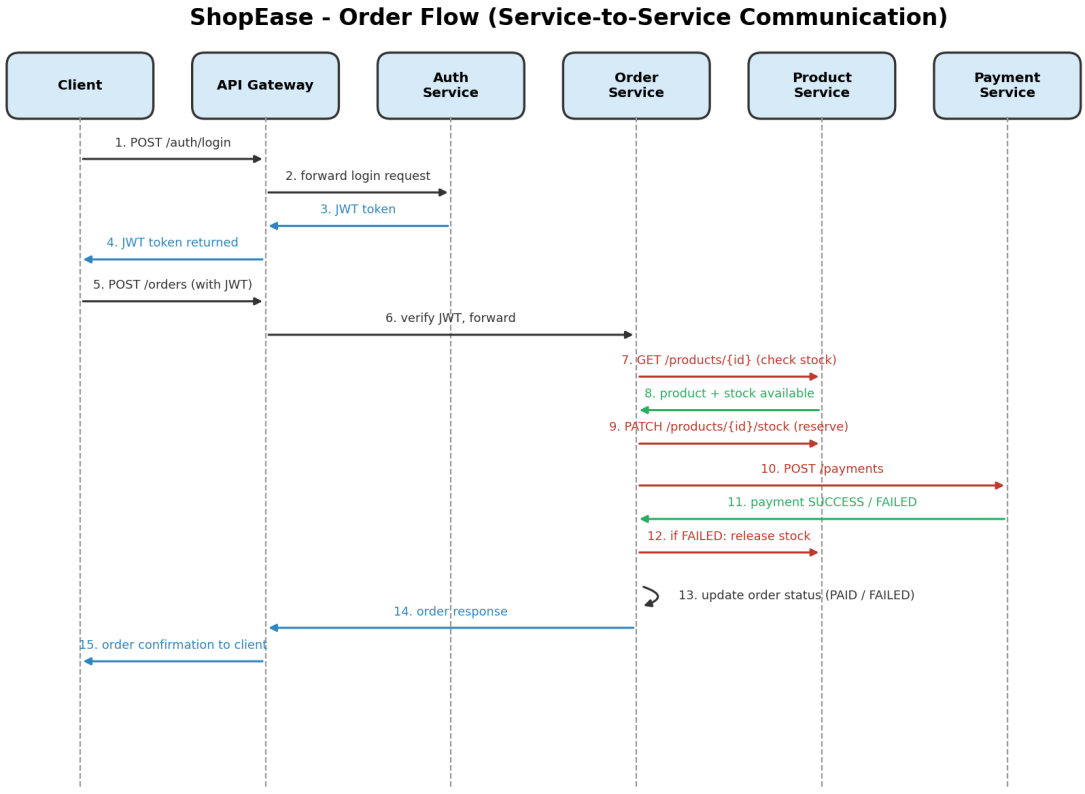
Prepared by: Dineshkumar S | Day 1 Deliverable

## 1. How Services Communicate

In ShopEase, services talk to each other using REST APIs over HTTP. A service never reads another service's database. For example, if the Order Service needs product details, it sends an HTTP request to the Product Service and gets a JSON response.

## 2. Complete Project Flow

Customer registration and login, JWT authentication, product browsing, order creation, product availability check, payment processing, order status update, and order history and reporting.




*Error case: if Product Service is down, Order Service returns 503 and the order is not created.*

## 3. Step by Step Flow

### Step 1: Registration and login

The customer registers using POST /auth/register. Then the customer logs in using POST /auth/login. The Authentication Service checks the email and password and returns a JWT token.

### Step 2: JWT authentication

The client keeps the token and sends it in the Authorization header with every next request. The API Gateway checks the token before forwarding the request. If the token is missing or wrong, it returns 401 Unauthorized.

### Step 3: Product browsing

The customer calls GET /products. The gateway routes the request to the Product Service, which returns the list of products.

### Step 4: Order creation

The customer places an order with POST /orders. The gateway sends it to the Order Service. The Order Service saves the order with the status PENDING.

### Step 5: Product availability validation

The Order Service calls GET /products/{id} on the Product Service for each item. If the stock is enough, the Order Service asks the Product Service to reduce the stock using PATCH /products/{id}/stock.

### Step 6: Payment processing

The Order Service calls POST /payments on the Payment Service with the order id and the amount. The Payment Service saves the payment and returns SUCCESS or FAILED.

### Step 7: Order status update

If the payment is SUCCESS, the Order Service changes the order status to PAID. If the payment is FAILED, it changes the status to FAILED and asks the Product Service to put the reserved stock back.

### Step 8: Order history and reporting

The customer can see all past orders using GET /orders and one order using GET /orders/{id}. The Order Service reads these from its own order_db.

## 4. Order Service Communication Summary

| Order Service calls | Endpoint | Why |
|---|---|---|
| Product Service | GET /products/{id} | Check the product and the stock |
| Product Service | PATCH /products/{id}/stock | Reduce stock, or restore it if the order fails |
| Payment Service | POST /payments | Process the payment |

## 5. Error Handling When a Service Is Unavailable

In a microservices system, a service can be down or slow at any time. The calling service must handle this properly instead of crashing.

### If the Product Service is down

The Order Service cannot check the stock. It should not create the order. It returns 503 Service Unavailable with a clear message, such as: "Product service is not available, please try again later."

### If the Payment Service is down

The stock was already reserved, so the Order Service must put the stock back by calling the Product Service. Then it marks the order as FAILED (or keeps it PENDING so the payment can be tried again) and returns 503 to the client.

### If the payment fails

The Order Service marks the order as FAILED and releases the reserved stock. The customer is told that the payment did not go through.

### Useful techniques

- Timeout: Do not wait forever for a service. Stop waiting after a few seconds (for example 5 seconds).
- Retry: Try again two or three times for temporary problems, with a short wait between tries.
- Circuit breaker: If a service keeps failing, stop calling it for a short time and return an error quickly.
- Clear error messages and logs: Always log which service failed, so it is easy to find the problem.
- Compensation: If a later step fails, undo the earlier step (for example release the stock).

### Error response example

```
HTTP/1.1 503 Service Unavailable
{
  "detail": "Product service is not available. Please try again later."
}
```

## 6. Conclusion

Services in ShopEase work together only through REST APIs. The Order Service is the main coordinator that talks to the Product and Payment services. Good error handling with timeouts, retries and undoing earlier steps keeps the system safe even when one service fails.